



Introduction to Computers and C++



1.6 C++

- ▶ C++ evolved from C, which was developed by Dennis Ritchie at Bell Laboratories.
- ▶ C
 - Available for most computers and is hardware independent.
 - It's possible to write C programs that are **portable** to most computers.
 - The widespread use of C with various kinds of computers (sometimes called **hardware platforms**) led to many variations.
 - American National Standards Institute (ANSI) cooperated with the International Organization for Standardization (ISO) to standardize C worldwide.
 - Joint standard document was published in 1990 and is referred to as *ANSI/ISO 9899: 1990*.



1.6 C++ (Cont.)

- ▶ C++, an extension of C, was developed at Bell Laboratories.
- ▶ C++ provides a number of features that “spruce up” the C language, but more importantly, it provides capabilities for object-oriented programming.



1.6 C++ (Cont.)

► C++ Standard Library

- C++ programs consist of pieces called **classes** and **functions**.
- Most C++ programmers take advantage of the rich collections of classes and functions in the **C++ Standard Library**.

Two parts to learning the C++ “world.”

- The C++ language itself, and
- How to use the classes and functions in the C++ Standard Library.

Many special-purpose class libraries are supplied by independent software vendors.



1.9 Typical C++ Development Environment (Cont.)

- ▶ C++ systems generally consist of three parts: a program development environment, the language and the C++ Standard Library.
- ▶ C++ programs typically go through six phases: edit, preprocess, compile, link, load and execute.



1.9 Typical C++ Development Environment (Cont.)

- ▶ Phase 1 consists of editing a file with an *editor* program, normally known simply as an editor.
 - Type a C++ program (**source code**) using the editor.
 - Make any necessary corrections.
 - Save the program.
 - C++ source code filenames often end with the `.cpp`, `.cxx`, `.CC` or `.C` extensions (note that `C` is in uppercase) which indicate that a file contains C++ source code.

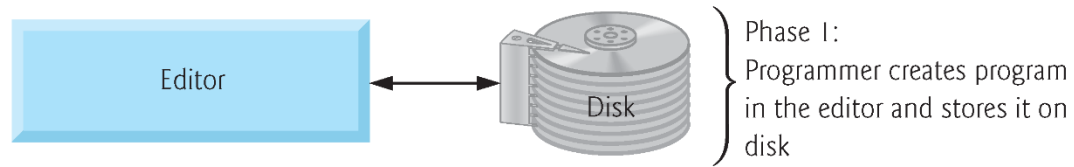


Fig. 1.6 | Typical C++ development environment—editing phase.



1.9 Typical C++ Development Environment (Cont.)

- ▶ Linux editors: `vi` and `emacs`.
- ▶ C++ software packages for Microsoft Windows such as Microsoft Visual C++ (microsoft.com/express) have editors integrated into the programming environment.
- ▶ You can also use a simple text editor, such as Notepad in Windows, to write your C++ code.
- ▶ **integrated development environments (IDEs)**
 - Provide tools that support the software-development process, including editors for writing and editing programs and debuggers for locating **logic errors**—errors that cause programs to execute incorrectly.



1.9 Typical C++ Development Environment (Cont.)

- ▶ Popular IDEs
 - Microsoft® Visual Studio
 - Dev C++
 - NetBeans
 - Eclipse
 - Apple's Xcode
 - CodeLite



1.9 Typical C++ Development Environment (Cont.)

- ▶ In phase 2, you give the command to **compile** the program.
 - A **preprocessor** program executes automatically before the compiler's translation phase begins (so we call preprocessing Phase 2 and compiling Phase 3).
 - The C++ preprocessor obeys commands called **preprocessing directives**, which indicate that certain manipulations are to be performed on the program before compilation.
 - These manipulations usually include other text files to be compiled, and perform various text replacements.

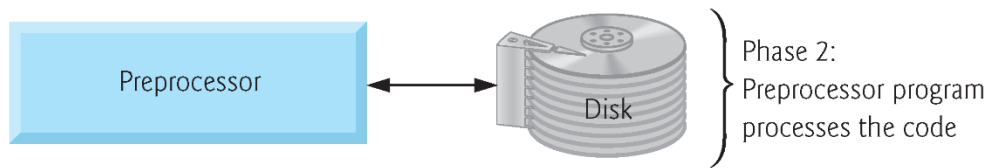


Fig. 1.7 | Typical C++ development environment—preprocessor phase.



1.9 Typical C++ Development Environment (Cont.)

- ▶ In Phase 3, the compiler translates the C++ program into machine-language code—also referred to as object code.

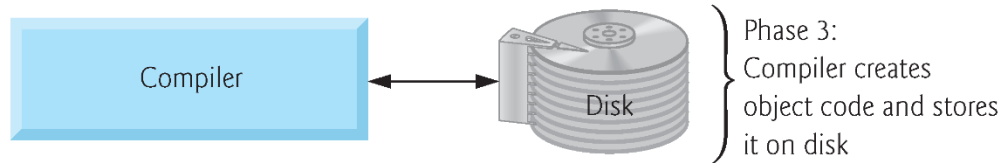


Fig. 1.8 | Typical C++ development environment—compilation phase.



1.9 Typical C++ Development Environment (Cont.)

- ▶ Phase 4 is called **linking**.
 - The object code produced by the C++ compiler typically contains “holes” due to these missing parts.
 - A **linker** links the object code with the code for the missing functions to produce an **executable program**.
 - If the program compiles and links correctly, an executable image is produced.

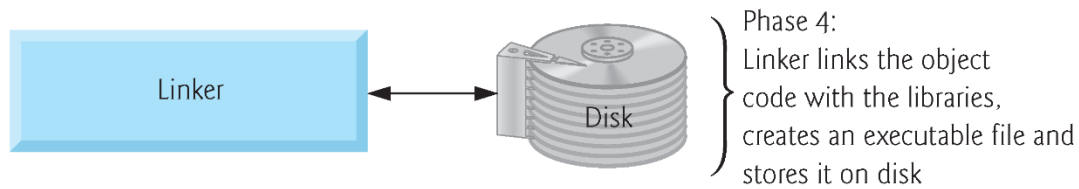


Fig. 1.9 | Typical C++ development environment—linking phase.



1.9 Typical C++ Development Environment (Cont.)

- ▶ Phase 5 is called **loading**.
 - Before a program can be executed, it must first be placed in memory. This is done by the **loader**, which takes the executable image from disk and transfers it to memory.

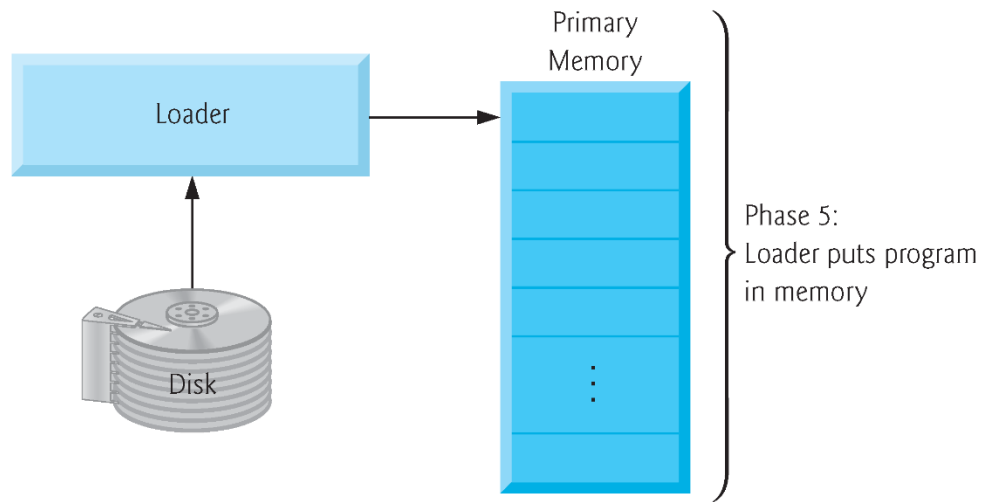


Fig. 1.10 | Typical C++ development environment—loading phase.



1.9 Typical C++ Development Environment (Cont.)

- ▶ Phase 6: Execution
 - Finally, the computer, under the control of its CPU, **executes** the program one instruction at a time.
 - Some modern computer architectures can execute several instructions in parallel.

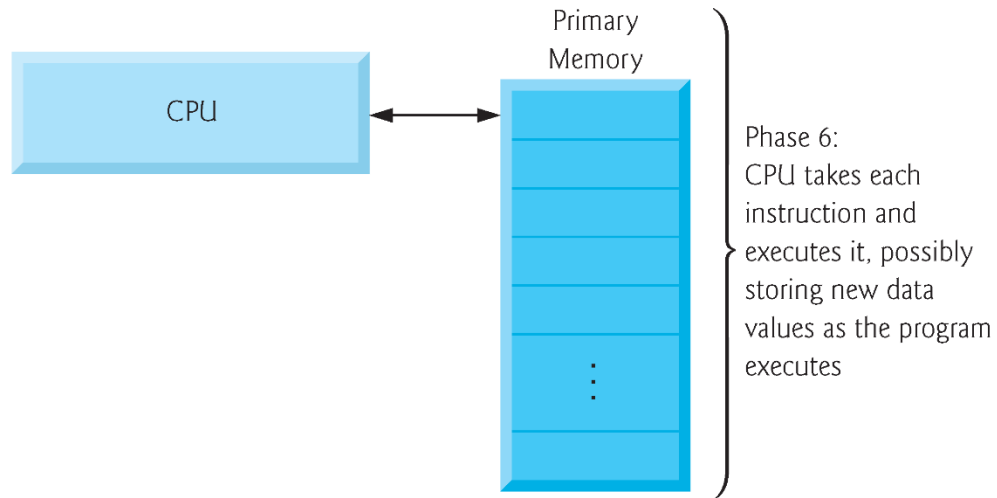


Fig. 1.11 | Typical C++ development environment—execution phase.



Data Hierarchy

- ▶ Data items processed by computers form a **data hierarchy** that becomes larger and more complex in structure as we progress from bits to characters to fields, and so on.

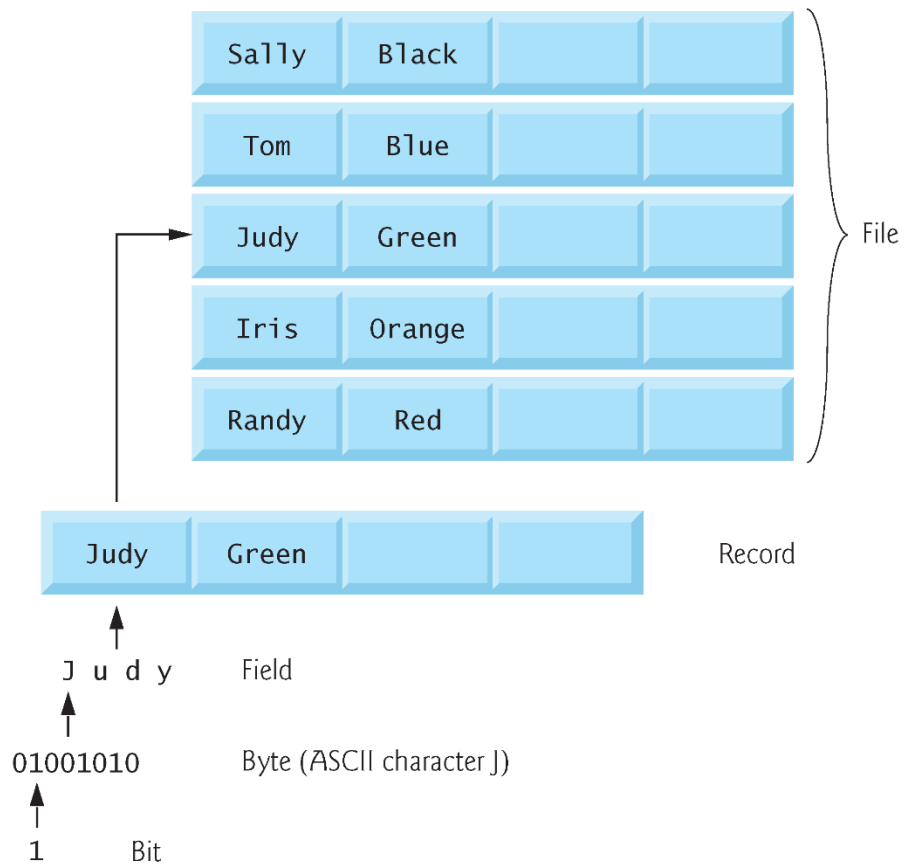


Fig. 1.3 | Data hierarchy.



Level	Description
Bits	The smallest data item in a computer can assume the value 0 or the value 1. Such a data item is called a bit (short for “binary digit”—a digit that can assume one of two values). It’s remarkable that the impressive functions performed by computers involve only the simplest manipulations of 0s and 1s— <i>examining a bit’s value</i> , <i>setting a bit’s value</i> and <i>reversing a bit’s value</i> (from 1 to 0 or from 0 to 1).
Characters	It’s tedious for people to work with data in the low-level form of bits. Instead, they prefer to work with <i>decimal digits</i> (0–9), <i>letters</i> (A–Z and a–z), and <i>special symbols</i> (e.g., \$, @, %, &, *, (,), –, +, ", :, ? and /). Digits, letters and special symbols are known as characters . The computer’s character set is the set of all the characters used to write programs and represent data items. Computers process only 1s and 0s, so every character is represented as a pattern of 1s and 0s. The Unicode [®] character set contains characters for many of the world’s languages. C++ supports several character sets, including 16-bit Unicode [®] characters that are composed of two bytes , each composed of eight bits. See Appendix B for more information on the ASCII (American Standard Code for Information Interchange) character set—the popular subset of Unicode that represents uppercase and lowercase letters, digits and some common special characters.

Fig. 1.4 | Levels of the data hierarchy. (Part I of 3.)



Level	Description
Fields	Just as characters are composed of bits, fields are composed of characters or bytes. A field is a group of characters or bytes that conveys meaning. For example, a field consisting of uppercase and lowercase letters could be used to represent a person's name, and a field consisting of decimal digits could represent a person's age.
Records	Several related fields can be used to compose a record. In a payroll system, for example, the record for an employee might consist of the following fields (possible types for these fields are shown in parentheses): • Employee identification number (a whole number) • Name (a string of characters) • Address (a string of characters) • Hourly pay rate (a number with a decimal point) • Year-to-date earnings (a number with a decimal point) • Amount of taxes withheld (a number with a decimal point) Thus, a record is a group of related fields. In the preceding example, all the fields belong to the same employee. A company might have many employees and a payroll record for each one.

Fig. 1.4 | Levels of the data hierarchy. (Part 2 of 3.)



Level	Description
Files	A file is a group of related records. [Note: More generally, a file contains arbitrary data in arbitrary formats. In some operating systems, a file is viewed simply as a <i>sequence of bytes</i> —any organization of the bytes in a file, such as organizing the data into records, is a view created by the application programmer.] It's not unusual for an organization to have many files, some containing billions, or even trillions, of characters of information.
Database	A database is an electronic collection of data that's organized for easy access and manipulation. The most popular database model is the relational database in which data is stored in simple <i>tables</i> . A table includes <i>records</i> and <i>fields</i> . For example, a table of students might include first name, last name, major, year, student ID number and grade point average. The data for each student is a record, and the individual pieces of information in each record are the fields. You can search, sort and manipulate the data based on its relationship to multiple tables or databases. For example, a university might use data from the student database in combination with databases of courses, on-campus housing, meal plans, etc.

Fig. 1.4 | Levels of the data hierarchy. (Part 3 of 3.)



1.5 Machine Languages, Assembly Languages and High-Level Languages

- ▶ Programmers write instructions in various programming languages, some directly understandable by computers and others requiring intermediate *translation steps*.
- ▶ These may be divided into three general types:
 - Machine languages
 - Assembly languages
 - High-level languages

1.5 Machine Languages, Assembly Languages and High-Level Languages (Cont.)



Machine Languages

- ▶ Any computer can directly understand only its own **machine language** (also called machine code), defined by its hardware architecture.
- ▶ Machine languages generally consist of numbers (ultimately reduced to 1s and 0s). Such languages are cumbersome for humans.

1.5 Machine Languages, Assembly Languages and High-Level Languages (Cont.)



Assembly Languages

- ▶ English-like *abbreviations* to represent elementary operations. These abbreviations formed the basis of **assembly languages**.
- ▶ *Translator programs* called **assemblers** were developed to convert early assembly-language programs to machine language.

1.5 Machine Languages, Assembly Languages and High-Level Languages (Cont.)



High-Level Languages

- ▶ To speed up the programming process further, high-level languages were developed in which single statements could be written to accomplish substantial tasks.
- ▶ Translator programs called **compilers** convert high-level language programs into machine language.
- ▶ Allow you to write instructions that look more like everyday English and contain commonly used mathematical expressions.

Compilers vs. Interpreters



- ▶ There are two general methods by which a program can be executed. It can be compiled, or it can be interpreted. Although programs written in any computer language can be compiled or interpreted, some languages are designed more for one form of execution than the other. For example, Java was designed to be interpreted, and C was designed to be compiled. However, in the case of C, it is important to understand that it was specifically optimized as a compiled language.



Compilers vs. Interpreters (cont.)

- ▶ In its simplest form, an interpreter reads the source code of your program one line at a time, performing the specific instructions contained in that line. This is the way earlier versions of BASIC worked. In languages such as Java, a program's source code is first converted into an intermediary form that is then interpreted. In either case, a run-time interpreter is still required to be present to execute the program.



Compilers vs. Interpreters (cont.)

- ▶ A compiler reads the entire program and converts it into *object code*, which is a translation of the program's source code into a form that the computer can execute directly. Object code is also referred to as *binary code* or *machine code*. Once the program is compiled, a line of source code is no longer meaningful in the execution of your program.



Compilers vs. Interpreters (cont.)

- ▶ In general, an interpreted program runs slower than a compiled program. Remember, a compiler converts a program's source code into object code that a computer can execute directly. Therefore, Compilation is a one-time cost, while interpretation incurs an overhead each time a program is run.